

Programação Web II

Apresentação da Disciplina

Período: 6º semestre

Carga horária: 60h

Código Classroom: cxuqp2it

Professor: Braully Rocha da Silva

Aula 02

APIs REST com Spring Boot

O que é REST?

REST — *Representational State Transfer*

Conjunto de **restrições de arquitetura** — não é um protocolo nem um

padrão

Pode ser implementado de **diversas maneiras**

Spring Boot + React 3

Programação Web II

Como funciona?

Uma solicitação a uma API RESTful transfere uma **representação do estado do recurso** para o solicitante (endpoint).

A representação é entregue via HTTP em vários formatos:

JSON (JavaScript Object Notation) — o mais usado
HTML, XML, texto simples, entre outros

O JSON é independente de linguagem e legível por **máquinas** e **humanos**. Spring

Boot + React 4

Programação Web II

Headers e parâmetros

Essenciais nos métodos HTTP por conterem identificadores dos **metadados** da

requisição:

Autorização

URI

Cache

Cookies

Cabeçalhos de solicitação e resposta carregam informações de conexão HTTP e **códigos de status**.

Para ser RESTful

Uma API deve atender aos seguintes critérios:

1. Arquitetura **cliente-servidor**
2. Comunicação **stateless**
3. Dados **armazenáveis em cache**
4. **Interface consistente**
5. **Sistema em camadas**

Critérios RESTful (1/2)

Cliente-servidor

Clientes, servidores e recursos, com solicitações gerenciadas por HTTP.

Stateless

Nenhum dado do cliente é armazenado entre requisições. Cada requisição é **independente**.

Critérios RESTful (2/2)

Interface consistente

Recursos identificáveis e distintos das representações

Representação contém as informações para o cliente manipular o recurso

Mensagens autoexplicativas

REST vs SOAP

	REST	SOAP
Natureza	Diretrizes flexíveis	Protocolo rígido
Formato	JSON (leve)	XML (mensageria)
Desempenho	Rápido, leve e escalável	Mais lento e pesado

Uso ideal	IoT e apps mobile	Transações com segurança integrada
------------------	-------------------	------------------------------------

Spring Boot + React 9

Programação Web II

Criando uma API REST manualmente

Exemplo de uma API REST simples em Java com Spring Boot:

```
@RestController
@RequestMapping("/api")
public class UsuarioController {
    @GetMapping("/usuarios")
    public List getAllUsuarios() {
        List users = new ArrayList<>();
        users.add(mapOf("nome", "João", "username", "joao123"));
        users.add(mapOf("nome", "Maria", "username", "maria456"));
    }
}
```

```
    }  
    return users;  
}
```

Exemplo de resposta JSON

```
[  
  {  
    "nome": "João",  
    "username": "joao123"  
  },  
  {  
    "nome": "Maria",  
    "username": "maria456"  
  }  
]
```

```
] }
```

JSON

O que é JSON?

JavaScript Object Notation (JSON) é um formato de **troca de dados** leve, baseado em texto, que é fácil de ler e escrever para humanos e fácil de analisar e gerar para máquinas.

As principais características do JSON incluem:

Estrutura de dados simples e legível

Suporte a tipos de dados primitivos (strings, números, booleanos)

Suporte a estruturas de dados complexas (objetos e arrays)

JSON é uma alternativa popular ao XML para a troca de dados entre sistemas. Spring Boot +

React 12

Programação Web II

Criando uma API REST com Spring Boot

Para expor uma API RESTful, o Spring Boot oferece:

Anotações para mapear endpoints HTTP

Serialização automática de objetos Java para JSON

Tratamento de exceções e **validação** de dados

Spring Boot + React 13

Programação Web II

Criar a classe de Usuario

```
public class Usuario {  
    private String nome;  
    private String username;  
    private String senha;
```

`private String email;`

`// Getters e Setters`

`}`

Spring Boot + React 14

Programação Web II

Substituir o método `getAllUsuarios`

`@GetMapping("/usuarios")`

`@GetMapping("/usuarios")`


```
public List<Usuario> getAllUsuariosWithClass() {  
    List<Usuario> users = new ArrayList<>();  
    users.add(new Usuario("João","joao123","senha123","joao@example.com")); users.add(new  
        Usuario("Maria","maria456","senha456","maria@example.com")); return users;  
}
```

Spring Boot + React 15

Programação Web II

Spring Boot

Os objetos Java são automaticamente convertidos para JSON pelo Spring Boot, graças à biblioteca Jackson.

Porém , para que a serialização funcione corretamente, é necessário garantir que a classe tenha **getters e setters** para todos os campos.

O que fazer quando precisamos **ocultar** algum campo, como a senha do usuário?

Utilizamos a anotação `@JsonIgnore` no campo que queremos ocultar. Criamos um **DTO (Data Transfer Object)** para controlar quais campos serão expostos na resposta da API.

Utilizamos a anotação `@JsonView` para definir diferentes visões de um mesmo objeto.

Spring Boot + React 16

Programação Web II

Exemplo de DTO

```
public class UsuarioDTO {
```

```
private String nome;  
private String username;  
private String email;
```

```
// Getters e Setters
```

```
}
```

Programação Web II

Spring Boot + React 17

JsonIgnore

```
public class Usuario {  
    private String nome;
```

```
private String username;  
@JsonIgnore  
private String senha;  
private String email;  
}
```

Para simplicidade, vamos utilizar JSONIgnore, mas em projetos reais, é recomendável utilizar DTOs para maior controle sobre os dados expostos.

Exercícios da Aula 02

Criar a Classe Permissao com os seguintes atributos:

id (Long)

nome (String)

descricao (String)

Criar o endpoint /permissoes com os métodos GET

Criar também uma Classe e endpoint do Seu Interesse para o projeto final.

Sugestão: Produto , Categoria , Pedido , Cliente , Fornecedor , Perfil , Comentario , Mensagem , Notificacao , Carro , etc.

Ao final, comitar e subir as alterações no repositório do GitHub.

Spring Boot + React 19

Programação Web II

Referências

Spring Boot Documentation

React Documentation

Material complementar da disciplina

[https://www.sp.senai.br/curso/desenvolvimento-de-api-rest-com-spring
boot/102669](https://www.sp.senai.br/curso/desenvolvimento-de-api-rest-com-spring-boot/102669)

<https://www.redhat.com/pt-br/topics/api/what-is-a-rest-api>

<https://spring.io/guides/tutorials/rest>

